

二叉树的应用 实验报告

姓名：时分

学号：PB23151799

设计目标

在C++环境下，利用二叉树与Huffman树的相关操作，实现对任意文件的压缩处理，压缩之后的文件后缀名为huff，同时可以对所有后缀名为huff的压缩文件进行解压缩。

以1个字节为单位进行Huffman编码，要求可以完成压缩和解压的步骤，并且解压出来的文件没有任何损失。

算法设计

压缩文件时执行的操作：

1. 读入文件，统计每种字节出现的次数（256种）
2. 根据字节出现的次数创建Huffman树
3. 由Huffman树得到各字节的Huffman编码
4. 将压缩前文件类型、各字节出现的次数信息写入.huff文件
5. 读入文件，按字节将对应的编码写入.huff文件，缺位用0补齐

解压缩文件时执行的操作：

1. 读入.huff文件，根据文件开头的信息获得Huffman树
2. 根据Huffman编码，将对应的字节写入文件

.huff文件结构为：

```
char FileType[FILE_TYPE_LENGTH] //源文件类型（,xxx）
int FileSize //源文件大小（用于解压时确定是否解码结束）
int count[256] //各字节按顺序的出现次数
//Huffman码比特流
```

基于贪心策略构建Huffman树的算法思想概括如下：

1. 给定n个权值作为n个叶子节点，构成初始的森林 $F=\{T_1, T_2, \dots, T_n\}$ ，其中每棵二叉树 T_i 仅有一个权值为 W_i 的根节点。
2. 在森林F中选取两棵根节点权值最小的树作为左右子树，构造一棵新的二叉树，并置新二叉树根节点的权值为其左右子树上根节点的权值之和。
3. 从森林F中删除这两棵被选中的树，同时将新构造的二叉树加入到森林F中。
4. 重复步骤2和步骤3，直到森林F中只含一棵二叉树为止，即得到Huffman树。

数据结构

采用结构体数组FileByteInfo[256]存储各字节（无符号字符）的信息，下标即表示无符号字符的值，结构体定义为：

```
typedef struct Info{
    int count;    //无符号字符出现的次数
    char *HCode;    //字符对应的Huffman编码
}Info;    //文件内无符号字符的信息
```

Huffman编码过程借助Huffman树实现，Huffman树采用顺序存储，其结点结构定义为：

```
typedef struct HTNode{
    unsigned char data;
    int weight;    //权重
    int lchild,rchild;
    int parent;
}HTNode,*HuffmanTree;
```

程序结构

各函数原型、功能与接口描述如下：

```
int menu();
    //输出菜单，返回操作
Status CountByte(FILE *OriginFile,Info *FileByteInfo,int &types,int &FileSize);
    //统计字符出现次数
Status FindSmallest(HuffmanTree T,int i,int &s1,int &s2);
    //找到数组中权重最小且未被连接过的两个结点
Status InitHuffmanTree(HuffmanTree &T,Info *FileByteInfo,int types);
    //建立Huffman树（顺序存储类型）
Status HuffmanCoding(HuffmanTree T,Info *FileByteInfo,int types);
    //根据Huffman树得到各字符的Huffman编码
Status ReNaming(char *FileName,char *FileType,int mode);
    //更改文件后缀
    //mode 1为更改为.huff后缀
    //mode 2为将.huff后缀改回原后缀
Status WriteInfo(FILE *NewFile,Info *FileByteInfo);
    //将各字节的出现次数写入文件
Status WriteHuffmanCode(FILE *OriginFile,FILE *NewFile,Info *FileByteInfo);
    //按源文件写入对应的Huffman编码
Status GetInfo(FILE *OriginFile,Info *FileByteInfo,int &types);
    //从文件中读入字符数信息
Status HuffmanDecoding(HuffmanTree T,int types,int FileSize,FILE
*OriginFile,FILE *NewFile);
    //按Huffman编码还原源文件
Status HuffmanDecompress(Info *FileByteInfo);
    //解压过程
```

调试分析

测试数据Input.txt采用样例文件：2_1.txt（英文文本），2_2.txt（中文文本），2_3.bmp（图片），2_4.mp4（视频），2_5.exe（可执行文件）。

发现存在如下问题：

1. 解码时无法正常结束，无法确定末尾的0是否是补位产生的。
解决方法：存储FileSize记录源文件大小，确定解码结束位置。
2. 解码时需要确定源文件类型。

解决方法：将原文件类型以字符串形式存入.huff文件开头。

算法的时空分析

各函数时空复杂度如下：

```
int menu();
    //输出菜单，返回操作
    //时间复杂度：O(1)
    //空间复杂度：O(1)
Status CountByte(FILE *OriginFile, Info *FileByteInfo, int &types, int &FileSize);
    //统计字符出现次数
    //时间复杂度：O(n)
    //空间复杂度：O(1)
Status FindSmallest(HuffmanTree T, int i, int &s1, int &s2);
    //找到数组中权重最小且未被连接过的两个结点
    //时间复杂度：O(n)
    //空间复杂度：O(1)
Status InitHuffmanTree(HuffmanTree &T, Info *FileByteInfo, int types);
    //建立Huffman树（顺序存储类型）
    //时间复杂度：O(n)
    //空间复杂度：O(n)
Status HuffmanCoding(HuffmanTree T, Info *FileByteInfo, int types);
    //根据Huffman树得到各字符的Huffman编码
    //时间复杂度：O(logn)
    //空间复杂度：O(1)
Status ReNaming(char *FileName, char *FileType, int mode);
    //更改文件后缀
    //mode 1为更改为.huff后缀
    //mode 2为将.huff后缀改回原后缀
    //时间复杂度：O(1)
    //空间复杂度：O(1)
Status WriteInfo(FILE *NewFile, Info *FileByteInfo);
    //将各字节的出现次数写入文件
    //时间复杂度：O(1)
    //空间复杂度：O(1)
Status WriteHuffmanCode(FILE *OriginFile, FILE *NewFile, Info *FileByteInfo);
    //按源文件写入对应的Huffman编码
    //时间复杂度：O(n)
    //空间复杂度：O(1)
Status GetInfo(FILE *OriginFile, Info *FileByteInfo, int &types);
    //从文件中读入字符数信息
    //时间复杂度：O(1)
    //空间复杂度：O(1)
Status HuffmanDecoding(HuffmanTree T, int types, int FileSize, FILE
*OriginFile, FILE *NewFile);
    //按Huffman编码还原源文件
    //时间复杂度：O(n)
    //空间复杂度：O(1)
Status HuffmanDecompress(Info *FileByteInfo);
    //解压过程
```

测试结果及分析

对五个文件，均可以在相同根目录下生成同名.huff文件，并可以独立从.huff文件解压生成源文件，其在文件资源管理器中的状态如下图：

 2_5.huff	2024/12/22 21:43	HUFF 文件	34 KB
 2_5.exe	2023/11/2 16:56	应用程序	53 KB
 2_4.mp4	2023/11/2 16:43	MP4 文件	128 KB
 2_4.huff	2024/12/22 21:43	HUFF 文件	128 KB
 2_3.huff	2024/12/22 21:43	HUFF 文件	35 KB
 2_3.bmp	2024/12/22 21:44	BMP 文件	142 KB
 2_2.txt	2023/11/2 16:19	文本文档	1 KB
 2_2.huff	2024/12/22 21:43	HUFF 文件	2 KB
 2_1.txt	2023/11/2 16:19	文本文档	1 KB
 2_1.huff	2024/12/22 21:43	HUFF 文件	2 KB

平均压缩率为61.8%，最优压缩率为24.6%。

实验体会与收获

通过本实验，我掌握了二叉树的递归特性，了解了二叉树的常用存储结构、创建及遍历等基本运算，并掌握了如何应用树的结构解决具体问题。